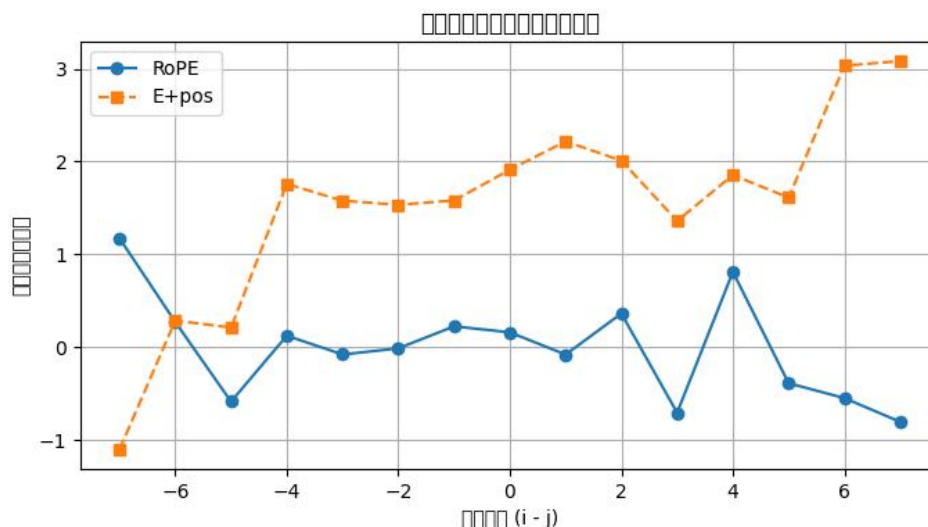




1. Transformer 为什么需要位置编码

Transformer 的自注意力机制是一个全连接操作，它看待输入序列时没有任何顺序概念。如果直接把词向量输入进去，模型无法区分“ABC”和“CBA”。位置编码的作用就是给每个位置的向量注入位置信息，让模型能够利用顺序关系。

2. 传统 Sinusoidal Position Encoding 是如何生成的

使用不同频率的正弦、余弦函数，为每个位置生成唯一的编码。代码中 `sinusoidal_position_encoding` 函数实现了完整的生成过程：偶数维度用 `sin`，奇数维度用 `cos`，频率沿维度指数衰减。这种编码的特点是相对位置可以通过线性变换由绝对位置表示，且能外推到训练时未见过的长度。

3. E + pos 的位置注入方式为什么有“内容和位置混合”的问题

在 E+pos 方式中，词向量和位置编码直接相加，然后共同参与后续的线性变换和点积注意力。这意味着词义信息（内容）和位置信息从最底层就纠缠在一起，模型必须学会将两者分离开。而且，这种加法引入的位置信息会随着层数和注意力头的变化而难以解耦，影响泛化性。

4. RoPE 不是加法，而是旋转

RoPE 不是将位置编码加到向量上，而是通过旋转矩阵对向量进行变换。具体来说，将向量每两个维度组成一个二维子空间，根据位置和频率计算旋转角度，然后分别在每个子空间内进行旋转。这样位置信息编码在向量的角度中，而向量的模长保持不变，保留了内容信息的强度。

5. RoPE 作用在 Q 和 K 上

在 Transformer 的每个自注意力层中，RoPE 仅施加在经过线性投影得到的 `query` 和 `key` 上（也可以施加在 `value` 上，但标准做法是 Q 和 K）。这确保了注意力计算中的点积能够反映相对位置。

6. RoPE 的点积天然包含相对位置

7. 对于经过 RoPE 变换的 query q_m 和 key k_n ，其点积可以分解为：

$$(R_m q) \cdot (R_n k) = q \cdot (R_{n-m} k)$$

其中 R_m 是对应位置 m 的旋转矩阵。由于旋转矩阵具有正交性和可加性，点积结果仅依赖于相对位置 $n-m$ ，而不是绝对位置 m 或 n 单独。代码最后的 `scores_same` 实验完美验证了这一点：当 query 和 key 内容完全相同时，注意力分数矩阵沿对角线完全相同，完全由相对位置决定。

7. Attention score 里的相对位置关系可以通过旋转结构自然出现

旋转操作将绝对位置信息转化为 query 和 key 之间的相位差，直接反映在点积结果中。因此模型不需要额外学习如何从加法噪声中分离出位置信息，相对位置关系自然编码在注意力分数中，使训练更高效，外推能力更强。

为什么 RoPE 比简单的 E+pos 更巧妙？

- 信息解耦：RoPE 只改变向量的方向，不改变模长，因此内容“强度”和位置“方向”是分开的。
- 相对位置直接编码：通过旋转变换的性质，注意力分数天然相对位置的函数，模型可以直接利用相对距离，而不需要从绝对位置编码中“推算”相对关系。
- 良好的外推性：旋转角度是相对位置的线性函数，通过调整频率，模型可以泛化到训练时未见过的更长序列。
- 减少学习负担：E+pos 需要模型在训练中学会如何处理内容与位置的混合信号，而 RoPE 直接将结构注入计算过程，使模型更容易学习。

前两个位置的前 4 维：

```
[[0.      1.      0.      1.      ]
 [0.84147098 0.54030231 0.09983342 0.99500417]]
```

原向量: [1. 0.] -> 旋转 90 度后: [0. 1.]

RoPE 输入形状: (5, 8) 输出形状: (5, 8)

E+pos 后向量 (位置 0 和位置 1 的前 4 维):

```
[[0.05683353 0.84984714 0.05807052 0.99814058]
 [0.97439183 0.57723516 0.20779777 1.05138878]]
```

RoPE 后向量 (位置 0 和位置 1 的前 4 维):

```
[[ 0.05683353 -0.15015286  0.05807052 -0.00185942]
 [ 0.04073952  0.13180394 -0.12947049  0.06387766]]
```

位置差矩阵 (i - j):

```
[[ 0 -1 -2 -3 -4 -5 -6 -7]
 [ 1  0 -1 -2 -3 -4 -5 -6]
 [ 2  1  0 -1 -2 -3 -4 -5]
 [ 3  2  1  0 -1 -2 -3 -4]
 [ 4  3  2  1  0 -1 -2 -3]
 [ 5  4  3  2  1  0 -1 -2]]
```

[6 5 4 3 2 1 0 -1]
[7 6 5 4 3 2 1 0]]

RoPE 注意力分数矩阵:

[[0.34 0.483 -0.672 0.195 -0.632 1.85 -1.061 1.173]
[0.7 2.27 -0.698 0.061 -0.573 0.051 -1.05 1.602]
[0.868 -1.512 1.425 0.696 0.896 0.882 0.489 -2.562]
[-2.708 1.59 0.395 -1.394 1.182 -2.745 0.971 0.583]
[0.742 -0.497 -1.379 -0.348 -1.065 1.859 0.755 -1.871]
[-0.753 0.852 -0.721 -0.135 -0.675 0.545 -1.858 1.623]
[-0.046 0.188 0.499 0.221 -0.237 -0.223 1.365 -0.098]
[-0.806 -1.053 -0.603 1.15 0.165 1.465 1.101 -2.211]]

E+pos 注意力分数矩阵:

[[-1.590e-01 1.870e-01 5.950e-01 1.352e+00 9.960e-01 1.834e+00
-2.590e-01 -1.104e+00]
[1.949e+00 3.407e+00 7.540e-01 2.140e+00 -3.060e-01 1.922e+00
9.210e-01 8.310e-01]
[3.000e+00 4.600e-02 4.386e+00 4.609e+00 4.820e+00 3.858e+00
4.114e+00 -2.122e+00]
[2.370e-01 1.292e+00 3.215e+00 4.220e-01 1.369e+00 -3.120e-01
1.520e+00 2.000e-03]
[1.027e+00 1.591e+00 9.110e-01 1.874e+00 -3.590e-01 2.120e+00
7.590e-01 1.463e+00]
[1.010e-01 1.580e-01 7.910e-01 8.090e-01 1.642e+00 2.682e+00
1.206e+00 1.201e+00]
[3.929e+00 2.515e+00 2.790e+00 1.886e+00 2.006e+00 2.379e+00
4.947e+00 8.150e-01]
[3.083e+00 2.137e+00 2.217e+00 3.438e+00 2.297e+00 4.042e+00
4.417e+00 -3.300e-02]]